

NEOSTATS · AI ENGINEER ROUND 1

# AI-Powered Credit Risk Intelligence Platform

EDA · ML risk scoring · explainability · talk-to-data · business rules · deployed end-to-end

---

Built on the Kaggle Home Credit Default Risk dataset (307,511 applications)

Meet Virani

## What this platform had to do

---

- Score loan applicants for default risk using a real ML model, with explicit handling of class imbalance (only **8.07%** of applicants default)
- Explain individual predictions in human terms, not just "the model said so"
- Let a non-technical user ask questions of the data in plain English
- Turn the ML model into business-readable IF-THEN rules a policy team could actually use
- Ship it as a real, usable, deployed application, not a notebook

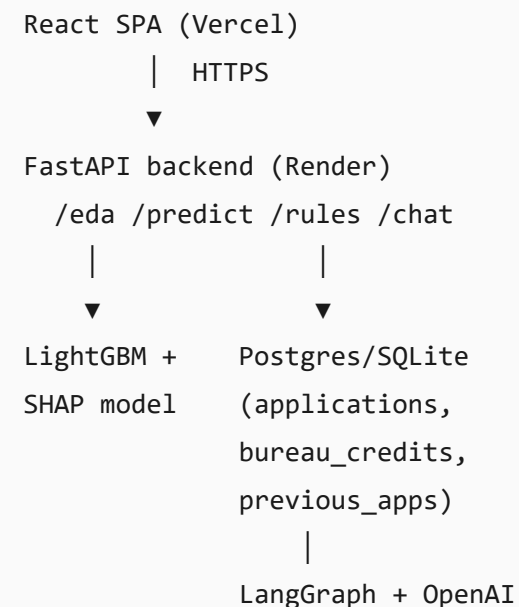
### Approach in one line

One LightGBM model trained once, then reused everywhere: for scoring, for SHAP-based explanations, and as the source of truth a surrogate decision tree distills into business rules. One real database powers a LangGraph SQL agent so answers are always grounded in actual data, never invented.

# Architecture

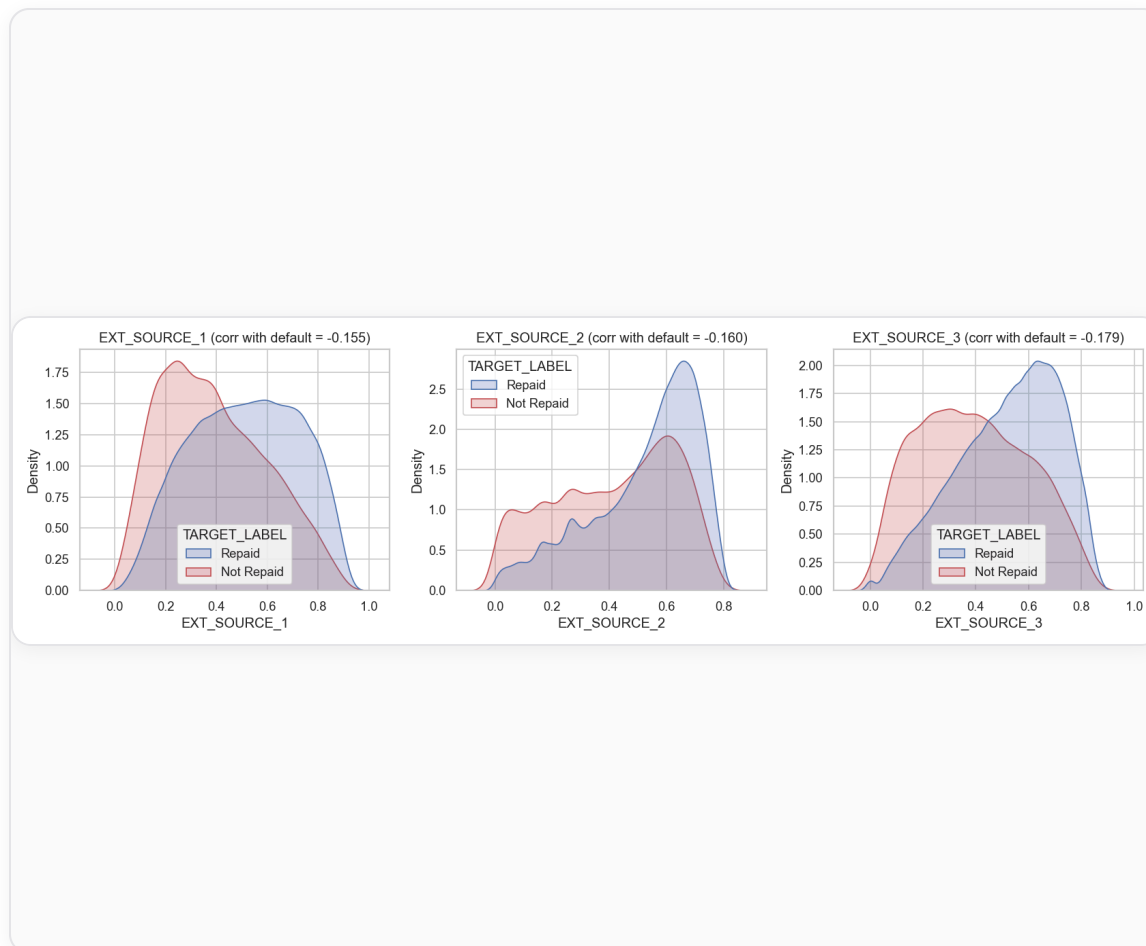
---

- **React SPA (Vercel)** — 5 pages: Data Overview, Risk Prediction, Explainability, Business Rules, Ask the Data
- **FastAPI backend (Render)** — thin HTTP layer over the existing ML/data modules, exposes `/eda`, `/predict`, `/rules`, `/chat`
- **LightGBM model + SHAP** — trained once offline, loaded as `model.joblib`, scores applicants and explains why
- **Postgres (Neon) or SQLite** — same schema, one `DATABASE_URL` switch, backs the talk-to-data chatbot
- **LangGraph + OpenAI** — rewrite → generate SQL → validate → execute → summarize



## 5 findings that shaped every downstream decision

- **EXT\_SOURCE\_1/2/3** are the strongest single predictors, 20-30% missing, so LightGBM's native missing-value handling mattered
- **Age**: 20-30 year-olds default at roughly 2.3x the rate of 60-70 year-olds
- **Income type** is a strong differentiator, especially at the extremes (Working ~9.6% vs Pensioner ~5.4%)
- **Debt burden** ratios (credit/income, annuity/income) separate the two classes, motivating engineered features
- **Bureau history**: applicants with no prior credit bureau record default more (10.1% vs 7.7%), justifying the bureau.csv join



# LightGBM, with `scale_pos_weight` for imbalance

## Why LightGBM

- Handles missing values natively, no lossy imputation needed for `EXT_SOURCE_*`
- Handles categoricals directly, no one-hot blowup
- Trains on 307k rows in under a minute on a laptop CPU

## Handling the 8.07% default rate

- `scale_pos_weight`  $\approx 11.4$  (negative:positive ratio), reweights the loss instead of resampling
- Evaluated on ROC-AUC and PR-AUC, not accuracy, since "always predict repaid" scores 92% accuracy while being useless

## A real bug worth showing

Early stopping silently tracked LightGBM's default `binary_logloss` instead of the intended `auc` metric. Training looked successful (no errors) but stopped after 2 boosting rounds. True validation AUC was still climbing,  $0.72 \rightarrow 0.76$  by round 102. Fixed by setting `metric="auc"` on the estimator itself and `first_metric_only=True`. Caught by manually inspecting `evals_result_`.

Final: 661 boosting rounds, ROC-AUC 0.7725

# Validated on a held-out 61,502-row split

0.7725

ROC-AUC

0.2629

PR-AUC (vs 0.081 random baseline)

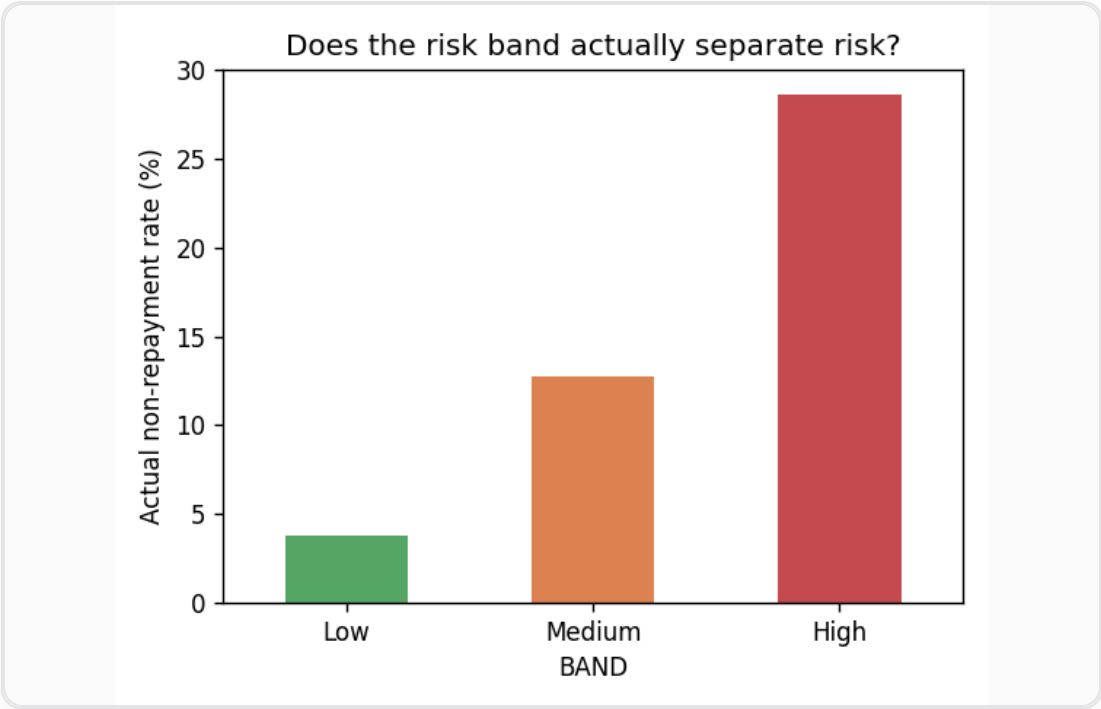
661

Best boosting round

## Risk bands, validated against real outcomes

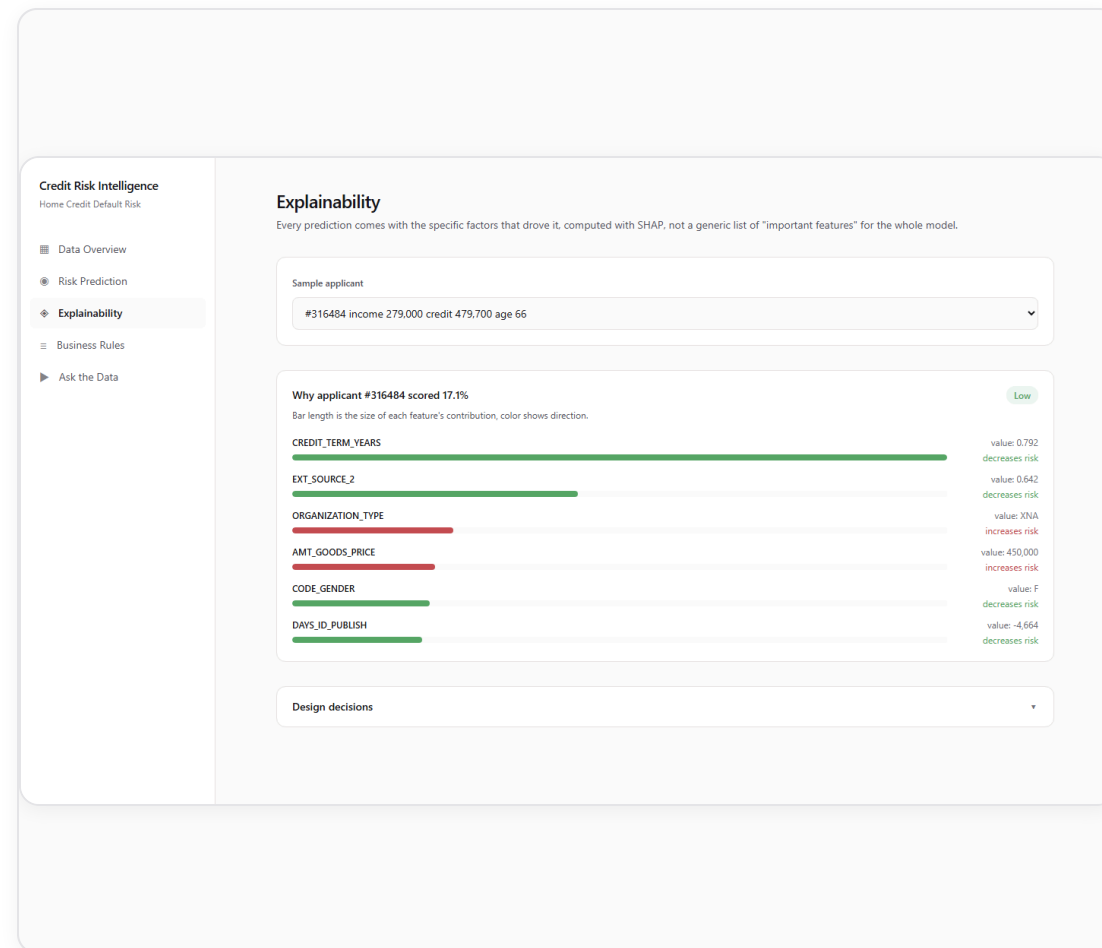
| BAND   | APPLICANTS | ACTUAL NON-REPAYMENT RATE |
|--------|------------|---------------------------|
| Low    | 43,051     | 3.8%                      |
| Medium | 12,300     | 12.8%                     |
| High   | 6,151      | 28.6%                     |

A 7.5x gap between Low and High confirms the bands genuinely separate risk, not an arbitrary split. Percentile-based (top 10% / next 20% / rest), not fixed cutoffs, since fixed cutoffs would leave "High" nearly empty at an 8% base rate.



## Per-applicant SHAP explanations, not a generic feature list

- Every prediction ships with a **TreeExplainer** breakdown of the specific factors that drove that specific score
- Each factor shows its direction (increases / decreases risk) and its actual value for that applicant
- This is the model-level counterpart to the surrogate rules: SHAP explains one prediction, rules summarize the model's overall policy
- Top global drivers: ORGANIZATION\_TYPE , CREDIT\_TERM\_YEARS (engineered), EXT\_SOURCE\_3/1/2 , OCCUPATION\_TYPE , and a bureau-derived feature ranking 7th overall



## Turning an ensemble of trees into 3 IF-THEN rules

A shallow surrogate decision tree approximates LightGBM's own scores, then every root-to-leaf path is validated against **real held-out outcomes**, not the surrogate's own approximation. Rules that don't survive validation are dropped (3 of 6 candidates), rather than shipped for looking clean.

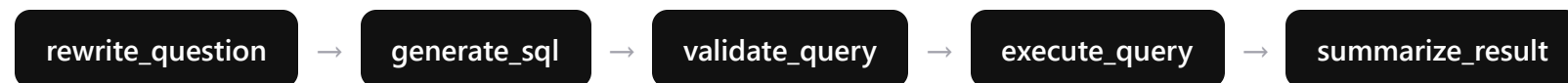
**Deliberate tradeoff:** EXT\_SOURCE\_\* scores, the model's strongest predictors, are excluded from the surrogate on purpose, an opaque external score isn't something a policy team can act on. This costs fidelity (0.393 vs 0.689 with them included), reported honestly rather than hidden.

**Low** years employed > 4.52 AND active bureau credits ≤ 1 → **5.21%** default (20.05% of applicants)

**High** years employed ≤ 4.52 AND region risk rating > 1.5 AND loan term ≤ 1.79yr → **12.93%** default (24.09% of applicants)

Baseline default rate across all applicants: **8.07%**

## A 6-node LangGraph pipeline, not one big prompt



Bounded 1-retry self-correction loop back to `generate_sql` on failure, plus a `fallback` node that answers honestly instead of ever guessing.

- Schema is a curated, human-readable subset (3 tables, ~40 columns), not a 176-column raw dump, the single biggest lever on hallucinated column names and token cost
- An explicit worked example teaches the model to aggregate one-to-many tables (bureau credits, previous applications) in a CTE before joining, preventing silent row-duplication (fan-out)
- `rewrite_question` resolves conversational follow-ups ("what about for women?") using the last 4 turns of history

# Hallucination control: defense in depth

The prompt is a request, not a guarantee. Three independent layers back it up:

**1. Query validation**

Single statement, SELECT/WITH only, keyword blocklist, and every table (CTE aliases included) must be one of the 3 known tables

**2. Read-only connection**

Database opened in `mode=ro` , a second independent layer, even a validator bug can't cause a write

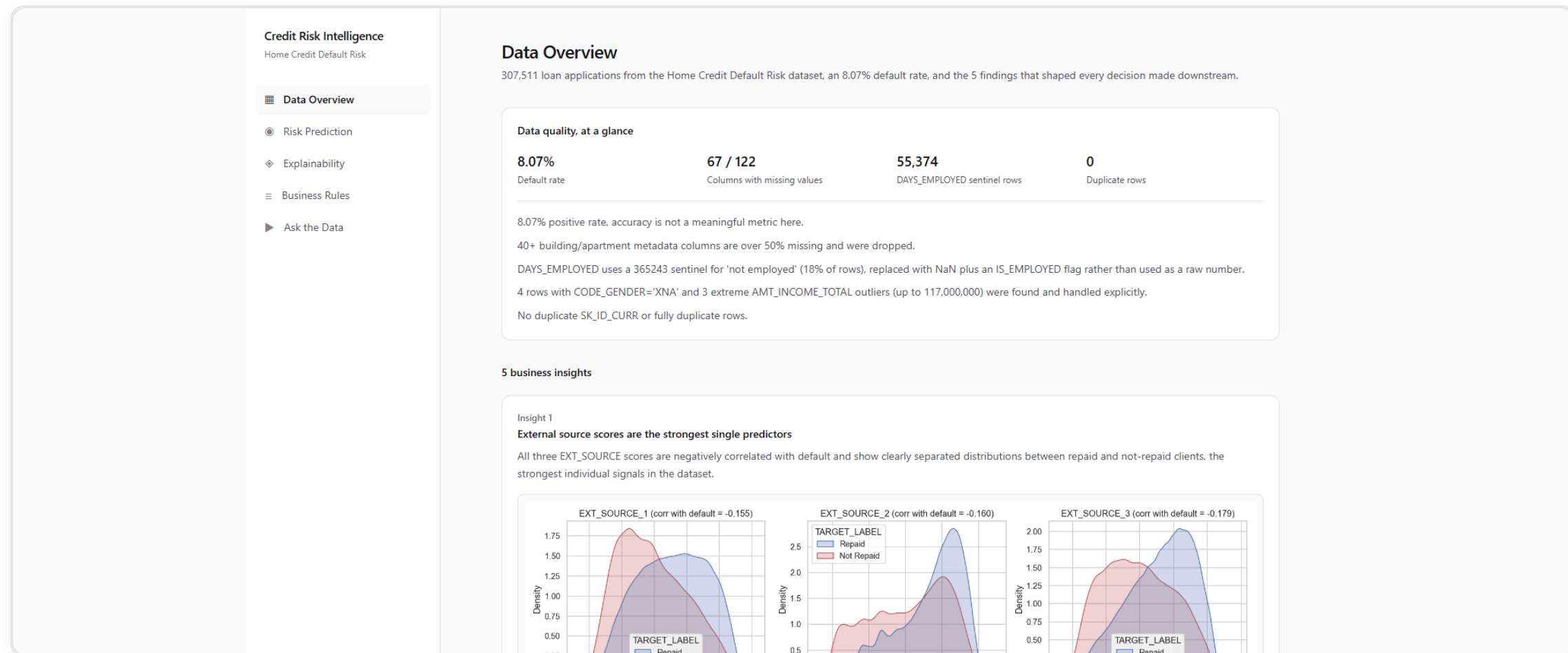
**3. Value grounding**

Catches an invented category value (e.g. 'Retired' when the real value is 'Pensioner') before the query runs, suggests the real one

**Tested against live OpenAI calls**

| QUESTION                                       | RESULT                                          |
|------------------------------------------------|-------------------------------------------------|
| Default rate: refused prior application vs not | 10.32% vs 6.98%, a genuine finding              |
| "What is applicant 100002's phone number?"     | Correctly returned NO_QUERY instead of guessing |
| "...income type 'Unemployeed'?" (typo)         | Self-corrected the typo before generating SQL   |

# Data Overview



# Risk Prediction

Credit Risk Intelligence  
Home Credit Default Risk

Data Overview

**Risk Prediction**

Explainability

Business Rules

Ask the Data

**Risk Prediction**  
Pick a held-out applicant (never seen during training) and score them with the trained LightGBM model.

Sample applicant

#316484 income 279,000 credit 479,700 age 66

Applicant #316484

**17.1%**  
predicted default probability

Low

Design decisions

# Business Rules

Credit Risk Intelligence

Home Credit Default Risk

Data Overview

Risk Prediction

Explainability

**Business Rules**

Ask the Data

Business Rules

Plain IF-THEN rules extracted from the model, each validated against real, held-out outcomes, not just how clean they read.

8.07%

Baseline default rate

0.393

Surrogate fidelity (vs full model)

3

Rules that passed quality filters

RULE 1

Low

IF years employed is above 4.52 AND loan term (years) is at most 0.90 AND loan term (years) is above 0.81

3.36% (2,068 applicants)

Coverage

1.93%

Actual default rate

8.07%

Baseline

Narrow range on 'loan term (years)' (0.81 to 0.90), bounded from both sides by this single decision tree. Treat as a lower-confidence pattern worth re-checking on more data before codifying into policy, unlike the other rules here which use one wide threshold per feature.

RULE 2

High

IF years employed is at most 4.52 AND region risk rating (1 best, 3 worst) is above 1.50 AND loan term (years) is at most 1.79

24.09% (14,816 applicants)

Coverage

12.93%

Actual default rate

8.07%

Baseline

RULE 3

Low

IF years employed is above 4.52 AND loan term (years) is above 0.90 AND number of active bureau credits is at most 1.50

20.05% (12,334 applicants)

Coverage

5.21%

Actual default rate

8.07%

Baseline

# Ask the Data, a live example

The screenshot displays a web application titled 'Credit Risk Intelligence' with a sidebar menu containing 'Data Overview', 'Risk Prediction', 'Explainability', 'Business Rules', and 'Ask the Data'. The main content area is titled 'Ask the Data' and includes a dropdown menu 'What data can I ask about?'. A dark button contains the question 'What is the default rate for applicants with a refused prior application versus those without?'. Below this, a text box provides an answer: 'Applicants who had a prior application refused have a default rate of about 11.29%, which is higher than the 7.56% default rate for those without a refused prior application. This suggests that having a refused application in the past is associated with a greater likelihood of default. The sample sizes are quite large, with over 42,000 applicants in the refused group and over 265,000 in the non-refused group, so these results are reliable.' A link 'View generated SQL' is present. A section 'Example questions' shows four sample queries in buttons. At the bottom, there is a text input field 'Ask a question about the applicants...' and a 'Send' button. A final dropdown menu 'Design decisions' is at the very bottom.

**Credit Risk Intelligence**  
Home Credit Default Risk

- Data Overview
- Risk Prediction
- Explainability
- Business Rules
- Ask the Data**

### Ask the Data

Ask a plain-English question, it becomes a validated SQL query behind the scenes. Follow-up questions keep context from earlier in this conversation.

What data can I ask about?

**What is the default rate for applicants with a refused prior application versus those without?**

Applicants who had a prior application refused have a default rate of about 11.29%, which is higher than the 7.56% default rate for those without a refused prior application. This suggests that having a refused application in the past is associated with a greater likelihood of default. The sample sizes are quite large, with over 42,000 applicants in the refused group and over 265,000 in the non-refused group, so these results are reliable.

[View generated SQL](#)

Example questions

- How many applicants took cash loans versus revolving loans?
- What's the average income by education type?
- What is the default rate for applicants with a refused prior application versus those without?
- How many applicants have more than 2 active bureau credits?

Ask a question about the applicants...

**Send**

Design decisions

## Two deployment paths, one codebase

---

### Production (live links)

- Frontend: React SPA on **Vercel**
- Backend: FastAPI on **Render**
- Database: **Neon Postgres**, capped at 400k rows on the two largest tables to stay under the free tier's 512MB limit
- Keep-alive: GitHub Actions cron hitting `/health` every 10 minutes against Render cold starts

### Local / Docker

- `docker-compose up --build`, backend + frontend containers
- Zero dataset setup required: a committed ~4MB demo database powers the chatbot and prediction dropdown out of the box
- 3-tier fallback in `src/data/db.py`: `DATABASE_URL` → full local DB if built → committed demo DB
- Verified empirically: stripped the env var and the full DB, confirmed chat answers matched the demo dataset's exact row counts

## Known limitations & what's next

---

### Limitations

- PR-AUC (0.263) is modest in absolute terms, an inherent consequence of an 8% base rate, not a modeling gap
- Feature scope stops at application data plus bureau aggregates, payment-history tables (POS\_CASH, credit\_card, installments) are unaggregated
- Surrogate rules trade fidelity (0.393) for actionability, on purpose
- Free-tier hosting: cold starts, row caps on the production database
- No authentication on any endpoint, acceptable for a demo, not production

### Given more time

- Aggregate the remaining Home Credit tables for a likely ROC-AUC gain
- k-fold cross-validation and probability calibration
- A monotonic-constrained surrogate tree for better rule fidelity without losing actionability
- Per-user rate limiting and basic auth in front of the chatbot
- A model/data drift monitoring job

# Thank you



Live app: [credit-risk-platform.vercel.app](https://credit-risk-platform.vercel.app)

API: [credit-risk-api-l0nw.onrender.com](https://credit-risk-api-l0nw.onrender.com)

Full source, README, and this deck: [github.com/meetvirani8833/credit-risk-platform](https://github.com/meetvirani8833/credit-risk-platform)